

MORAA GemVision

Work Audit Log · Phase 3 Runtime Hardening · generated 2026-10-02

How to use this page: tick the box ☐ at the start of a task once you have checked that it is done and done well. Write the status (done, in progress, blocked, not started) by hand in the empty box at the right of each task, and fill in the summary table below. Nothing on this page is pre-filled with a status. The reasons, decisions (D1, D2...) and errors (E1, E2...) are explained once for the whole phase, after the checklist; tasks refer to them by number.

Domain	Tasks	Done	In progress	Blocked	Not started
1. Event loop and database	3				
2. Image providers	4				
3. Proof	1				
Total	8				

Why this phase

The web server handles everything in one process on one event loop. A slow image provider, a heavy photo decode, a database call or a retry storm could freeze every customer's messages, strand paid orders, or burn the daily spending limit on failed calls. The audit predicted these failures under a burst of 100 requests a minute. Without this phase one slow minute at one provider could make the whole bot unresponsive and customers could pay for orders that never complete.

WHAT WAS DONE, IN SHORT

Slow or heavy work moved off the main loop into separate worker pools; database connections are given back before any long wait; every outside call has a deadline and a sensible retry rule; a shared daily counter in the database caps AI spending across all processes and gives slots back when a call fails; and a repeatable load test with eleven scenarios proves each fix.

Checklist

1. Event loop and database

One customer's heavy work never freezes the others.

- ☐ **1.1 Photo decoding and other CPU work run in their own worker pool instead of on the main loop**
- FILES backend/app/utils/executors.py, backend/app/services/meta_whatsapp_service.py
- HOW TO CHECK Run the load scenario h (tests/load_scenarios/run_load_scenarios.py --only h). Expect PASS and a worst single stall under 100 ms.
- ☐ **1.2 Database connections are given back before any long network wait**
- FILES backend/app/api/routes/meta_webhook.py, backend/app/services/meta_whatsapp_service.py
- HOW TO CHECK Run scenario k: 30 simultaneous orders finish without a connection-pool timeout.
- ☐ **1.3 Database-only routes no longer block the loop (plain functions run in the thread pool)**
- FILES backend/app/api/routes/analysis.py, backend/app/api/routes/history.py, backend/app/api/routes/reports.py
- HOW TO CHECK Open the dashboard history page while an order is generating: it stays responsive.

2. Image providers

Slow or failing providers cost time, not money or availability.

- ☐ **2.1 The daily AI limit is shared by all server processes and failed calls give their slot back**

FILES backend/app/services/spend_counter.py, backend/app/models/generation_spend.py, backend/alembic/versions/0013*.py

HOW TO CHECK Make a provider fail on purpose: the day's count does not rise. Run two server processes: together they stop at the one limit.

- ☐ **2.2 A rate limit is retried with waits then falls back; a billing or daily-quota stop halts and refunds**

FILES backend/app/ai/image_generation_manager.py

HOW TO CHECK Unit tests in tests/test_provider_hardening.py show both behaviours; no live provider was used.

- ☐ **2.3 A whole Catalog Pack cannot be held forever by one stuck image**

FILES backend/app/services/meta_whatsapp_service.py, backend/app/config.py

HOW TO CHECK Scenario d (a provider that hangs): the pack still delivers its finished images and refunds only if none finished.

- ☐ **2.4 Every outside HTTP call has a timeout; the Razorpay and Meta clients are shared and reused**

FILES backend/app/ai/providers/, backend/app/services/razorpay_service.py

HOW TO CHECK Unit tests in tests/test_external_calls.py. Not exercised against live Meta or Razorpay.

3. Proof

- ☐ **3.1 A repeatable load test with eleven scenarios (a-k) passes and is locked**

FILES backend/tests/load_scenarios/

HOW TO CHECK Run backend/venv-local/Scripts/python.exe

backend/tests/load_scenarios/run_load_scenarios.py. Expect a-i and k PASS. Scenario h measures timing and can vary on a busy computer; rerun it alone if it fails.

Key decisions and why

D1 Use separate worker pools for CPU work (photo decoding), database/file work and network sends, sized from the machine's cores.

WHY THIS CHOICE

One shared pool let slow network calls starve photo decoding and vice versa. Separate pools keep each kind of work from blocking the others, and the load test showed the longest loop stall dropping from 2.7 seconds to under 0.1 second.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Python's default thread pool** — not chosen because: Unbounded mixing of work types; measured stalls up to 2.7 seconds.
- **Separate processes** — not chosen because: Much more memory and complexity for the same benefit at this size.

Relates to task(s): 1.1, 1.2

D2 Count the daily AI ceiling in a database table shared by every server process, with a guarded update, and give slots back when a call fails.

WHY THIS CHOICE

An in-memory counter is per process (two processes = double the ceiling) and was never given back after failures, so failed orders ate the limit.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Redis counter** — not chosen because: Needs a new service the owner has not chosen yet; the database already exists.
- **Per-process counter divided by process count** — not chosen because: Wrong whenever processes restart or differ in load.

Relates to task(s): 2.1

D3 Separate a per-minute rate limit (retry, then fall back) from billing or daily-quota exhaustion (halt, refund, do not fall back).

WHY THIS CHOICE

The same Gemini wording appears for both. Treating a short rate limit as 'out of money' made orders fail needlessly; treating a real quota exhaustion as retryable burned the second provider's money.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Treat every 429 as retryable** — not chosen because: Would hammer a provider with an exhausted account.
- **Treat every 429 as fatal** — not chosen because: Failed orders that a ten-second wait would have served.

Relates to task(s): 2.2

D4 Bound only the GENERATION part of a Catalog Pack with a deadline; a style that finished in time is always uploaded and delivered.

WHY THIS CHOICE

A deadline over the whole job could cut a finished image off in the middle of its upload, wasting a paid call.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **One deadline for generation plus upload** — not chosen because: Cancels finished work.

Relates to task(s): 2.3

Errors faced

E1 TimeoutError: QueuePool limit of size 5 overflow 10 reached, connection timed out

Cause: A product-choice handler held a database connection while it waited on another thread that needed its own, so bursts of 16 or more orders ran out of connections.

Fix: End the transaction before the wait. Load scenario k now proves 30 simultaneous orders complete.

Task: 1.3

E2 AttributeError: 'Depends' object has no attribute 'append' (load harness)

Cause: The harness called the webhook function directly, so the tracking dependency was a placeholder object.

Fix: The webhook tolerates being called without the dependency (duplicate protection still works). A test-harness workaround.

Task: 3.1

E3 Reviewer: refund could be skipped when a 5xx came after the provider had accepted a send

Cause: Message sends were retried on 5xx; a retry after a half-delivered send could duplicate it.

Fix: Sends retry only on 429 and connection failures, never on 5xx.

Task: 2.4